



# COLLEGE DE PARIS

## Projet 4 :

Réseaux feedforward (MLP) – Reconnaissance de chiffres.

**PRESENTE ET REDIGE PAR : ABALO KOMI OLIVIER NESTOR**

**AKOLLY SEVI MARCEL**

**AKOLLY ASSO MARCELLIN**

**FILIERE-NIVEAU : IABD-B3**

**MODULE : INTRODUCTION AU DEEP LEARNING**

**PROFESSEUR: Dr AFENYIVÉH Dzo M. Serge**

**UNIVERSITE : COLLEGE DE PARIS SUPERIEUR LOME TOGO**

**ANNEE ACADEMIQUE : 2025-2026**

## TABLE DES MATIÈRES

|                                               |    |
|-----------------------------------------------|----|
| Résumé (Abstract) .....                       | 3  |
| 1. Introduction .....                         | 3  |
| 1.1 Contexte .....                            | 3  |
| 1.2 Problématique .....                       | 3  |
| 1.3 Objectif du projet .....                  | 4  |
| 1.4 Lien avec le cours .....                  | 4  |
| 2. Présentation du dataset .....              | 5  |
| 2.1 Description générale .....                | 5  |
| 2.2 Structure des données .....               | 5  |
| 2.3 Chargement des données avec PyTorch ..... | 5  |
| 3. Exploration et analyse des données .....   | 5  |
| 3.1 Visualisation des images .....            | 5  |
| 3.2 Distribution des classes .....            | 6  |
| 3.3 Observations .....                        | 6  |
| 4. Prétraitement des données .....            | 6  |
| 4.1 Normalisation .....                       | 6  |
| 4.2 Aplatissement des images .....            | 6  |
| 4.3 Création des DataLoaders .....            | 7  |
| 5. Méthodologie .....                         | 7  |
| 5.1 Modèle baseline .....                     | 7  |
| 5.2 Modèle MLP .....                          | 7  |
| 5.3 Calcul du nombre de paramètres .....      | 7  |
| 5.4 Fonction de perte .....                   | 8  |
| 5.5 Optimiseur .....                          | 8  |
| 5.6 Entraînement .....                        | 8  |
| 6. Expérimentations .....                     | 9  |
| 7. Évaluation des performances .....          | 9  |
| 7.1 Accuracy .....                            | 9  |
| 7.2 F1-score .....                            | 9  |
| 7.3 Matrice de confusion .....                | 10 |

|                                       |           |
|---------------------------------------|-----------|
| <b>8. Analyse des résultats .....</b> | <b>10</b> |
| <b>8.1 Interprétation .....</b>       | <b>10</b> |
| <b>8.2 Forces et limites .....</b>    | <b>10</b> |
| <b>8.3 Analyse des erreurs .....</b>  | <b>10</b> |
| <b>9. Améliorations .....</b>         | <b>11</b> |
| <b>10. Conclusion .....</b>           | <b>11</b> |
| <b>11. Références .....</b>           | <b>11</b> |

## **RÉSUMÉ (ABSTRACT)**

Dans ce projet, nous abordons un problème fondamental du Deep Learning : la reconnaissance des chiffres manuscrits 0 - 9. L'objectif est de concevoir un modèle capable de classer des images représentant des chiffres de 0 à 9. Pour cela, nous utilisons le dataset MNIST, composé de 70 000 images en niveaux de gris. Nous mettons en œuvre un modèle de type MLP (Multi-Layer Perceptron) en utilisant la bibliothèque PyTorch. Après une phase d'exploration et de prétraitement des données, nous construisons 2 architectures de réseaux neuronaux (modèle simple et MLP) afin de comparer leurs performances. Les résultats obtenus montrent que le MLP permet d'atteindre une excellente accuracy (> 97%), bien que certaines erreurs persistent sur des chiffres visuellement similaires. Ce projet met en évidence les forces et limites des réseaux feedforward et ouvre la voie à des modèles plus avancés comme les CNN.

## **1. INTRODUCTION**

### **1.1 Contexte**

La reconnaissance de chiffres manuscrits est un problème classique en intelligence artificielle. Elle est utilisée dans de nombreux domaines tels que : la lecture automatique des chèques bancaires, la numérisation de documents, les systèmes postaux... Avec l'essor du Deep Learning, les réseaux neuronaux sont devenus les outils les plus performants pour traiter ce type de données.

### **1.2 Problématique**

Le problème consiste à répondre à la question suivante : « Comment reconnaître automatiquement les chiffres manuscrits 0 - 9 à partir d'une image avec un ordinateur? » Ce problème est difficile car les écritures varient selon les individus, certaines images sont bruitées et certains chiffres se ressemblent (ex : 3 et 5, 4 et 9).

### **1.3 Objectif du projet**

L'objectif est de :

- construire un modèle de type MLP (Multi-Layer Perceptron) adapté à la classification multiclasse,
- entraîner ce modèle sur le dataset MNIST composé de 60 000 images d'entraînement,
- évaluer les performances du modèle à l'aide de différentes métriques (accuracy, F1-score, matrice de confusion),
- analyser les erreurs de prédiction afin de comprendre les limites du modèle et ses cas de confusion.

Ce projet permet ainsi de suivre un pipeline complet de Deep Learning, allant de la préparation des données jusqu'à l'évaluation des performances.

### **1.4 Lien avec le cours**

Ce projet s'inscrit directement dans le cadre du cours Introduction au Deep Learning, plus précisément dans la partie dédiée aux réseaux de neurones feedforward. Il met en pratique les notions suivantes :

- les réseaux feedforward : qui constituent la base des réseaux de neurones artificiels,
- les perceptrons multicouches (MLP) : utilisés pour la classification de données structurées ,
- la propagation avant (forward propagation) : permettant de calculer les sorties du réseau,
- les couches denses (fully connected layers) : qui assurent la transformation progressive des représentations,
- le calcul des paramètres du réseau : essentiel pour comprendre la complexité du modèle.

Ainsi, ce projet porte directement sur la construction et l'analyse d'un perceptron multicouche (MLP) appliqué au dataset MNIST, en étudiant ses couches denses, son mécanisme de propagation avant, ainsi que le nombre total de paramètres appris par le modèle.

## **2. PRÉSENTATION DU DATASET**

### **2.1 Description générale**

Le dataset MNIST est une base de données standard en Deep Learning contenant :

- ❖ 60 000 images d'entraînement,
- ❖ 10 000 images de test,
- ❖ des images de taille 28 x 28 pixels,
- ❖ 10 classes (chiffres de 0 à 9).

## **2.2 Structure des données**

Chaque image est en niveaux de gris, représentée par une matrice  $28 \times 28$ , convertible en vecteur de 784 valeurs et possède un label correspondant au chiffre représenté.

## **2.3 Chargement avec PyTorch**

```
# Chargement des données MNIST
train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

test_dataset = datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)
```

Ce code permet de télécharger automatiquement le dataset et de convertir les images en tenseurs exploitables.

## **3. EXPLORATION ET ANALYSE DES DONNÉES**

### **3.1 Visualisation**

La visualisation des images permet de vérifier la qualité des données.

```
# Affichage de quelques images
for i in range(5):
    image, label = train_dataset[i]
    plt.imshow(image.squeeze(), cmap='gray')
    plt.title(f"Label: {label}")
    plt.axis('off')
    plt.show()
```

### **3.2 Distribution des classes**

Il est important de vérifier si les classes sont équilibrées.

Observation : les 10 classes sont globalement équilibrées et cela facilite l'apprentissage.

### **3.3 Observations**

- Les chiffres sont centrés.
- Les écritures varient fortement.

- Certaines images sont difficiles à interpréter même pour un humain.

## **4. PRÉTRAITEMENT DES DONNÉES**

### **4.1 Normalisation**

Dans le dataset MNIST, chaque pixel d'une image possède une valeur comprise entre 0 et 255, où : 0 = noir et 255 = blanc.

Problème : ces valeurs sont trop grandes et non homogènes pour un réseau de neurones.

Solution : on effectue une normalisation, c'est-à-dire qu'on ramène les valeurs entre 0 et 1 :

$$x_{normalisé} = \frac{x}{255}$$

On normalise pour accélérer l'apprentissage, stabiliser la descente de gradient, éviter les oscillations de la loss et améliorer la convergence du modèle.

```
# Transformation : normalisation automatique (0 → 1)
transform = transforms.Compose([
    transforms.ToTensor()
])
```

Important : ToTensor() fait déjà la division par 255 automatiquement.

### **4.2 Aplatissement des images**

Un MLP ne prend pas des images en 2D mais des vecteurs.

```
images = images.view(-1, 28*28)
```

Cela transforme une image 28 × 28 en vecteur de 784 valeurs.

### **4.3 DataLoader**

```
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

Elle permet de gérer les batches et d'accélérer l'entraînement.

## **5. MÉTHODOLOGIE**

### **5.1 Modèle baseline**

Un modèle simple permet d'avoir une référence.

```
# Modèle simple
model_simple = nn.Sequential(
    nn.Linear(784, 10)
)
```

Exemple : un modèle avec une seule couche cachée.

## **5.2 Modèle MLP**

```
# Modèle MLP
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(784, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10)
        )

    def forward(self, x):
        return self.model(x)

model = MLP()
```

## **5.3 Calcul des paramètres**

Couche 1 :  $(784 \times 128) + 128 = 100\,480$ ,

Couche 2 :  $(128 \times 64) + 64 = 8\,256$ ,

Couche 3 :  $(64 \times 10) + 10 = 650$ ,

Total = 109 386 paramètres.

```
def count_params(model):
    return sum(p.numel() for p in model.parameters())

print("Params Simple:", count_params(model_simple))
print("Params MLP:", count_params(model))
```

---

```
Params Simple: 7850
Params MLP: 109386
```

## **5.4 Fonction de perte**

```
criterion = nn.CrossEntropyLoss()
```

Elle est adaptée à la classification multiclasse.

## **5.5 Optimiseur**

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

Il permet d'ajuster les poids.

## **5.6 Entraînement (Plusieurs époques, Mise à jour des poids et Minimisation de la loss)**

```
def train_model(model):
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    losses_per_epoch = [] # Initialiser la liste pour stocker les pertes

    for epoch in range(3):
        total_loss = 0

        for images, labels in train_loader:
            images = images.view(images.size(0), -1)

            outputs = model(images)
            loss = criterion(outputs, labels)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
        losses_per_epoch.append(total_loss) # Ajouter la perte totale pour l'époque
        print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}")
    return losses_per_epoch # Retourner la liste des pertes

print("Training Simple Model")
train_model(model_simple)

print("\nTraining MLP Model")
train_model(model)
```

---

```
Training Simple Model
Epoch 1, Loss: 508.3533
Epoch 2, Loss: 303.4665
Epoch 3, Loss: 277.0054
```

```
Training MLP Model
Epoch 1, Loss: 319.6839
Epoch 2, Loss: 135.7489
Epoch 3, Loss: 94.4012
[319.68389904499054, 135.7488916888833, 94.40119456779212]
```

## **6. EXPÉRIMENTATIONS**

Nous testons plusieurs architectures :

Modèle simple (1 couche)

```
model = model_simple
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

---

```
Sequential(
  (0): Linear(in_features=784, out_features=10, bias=True)
)
```

Modèle MLP (3 couches)

```
model = MLP()
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

---

```
MLP(
  (model): Sequential(
    (0): Linear(in_features=784, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=64, bias=True)
    (3): ReLU()
    (4): Linear(in_features=64, out_features=10, bias=True)
  )
)
```

Résultat : plus le modèle est profond (3 couches), meilleure est l'accuracy (jusqu'à un certain point).

## **7. ÉVALUATION DES PERFORMANCES**

### **7.1 Accuracy**

L'accuracy (ou exactitude) représente le pourcentage total de bonnes prédictions faites par le modèle.

$$Accuracy = \frac{\text{Nombre de prédictions correctes}}{\text{Nombre total de prédictions}}$$

**Interprétation dans MNIST** : dans notre projet chaque image appartient à une seule classe (0 à 9) donc l'accuracy mesure directement la performance globale du MLP (si 9700 images sur 10000 sont bien classées : Accuracy=97%).

**Limite** : l'accuracy peut être trompeuse si certaines classes sont mal reconnues et les erreurs sont concentrées sur quelques chiffres donc elle ne suffit pas seule.

### **7.2 F1-SCORE**

Le F1-score est une moyenne harmonique entre précision (qui mesure la qualité des prédictions positives) et recall (qui mesure la capacité du modèle à retrouver tous les exemples d'une classe).

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

On utilise le F1-score parce que la précision seule et le rappel seuls ne sont pas suffisants, le F1-score les équilibre.

**Interprétation dans MNIST :** un bon modèle doit avoir une précision élevée et un rappel élevé, donc un F1-score élevé. Dans MNIST, F1-score global  $\approx 0.97$  (très bon modèle).

### 7.3 Matrice de confusion

La matrice de confusion est un tableau qui compare les vraies classes et les classes prédites. Elle permet d'identifier les erreurs.

**Interprétation :** diagonale principale = bonnes prédictions et hors diagonale = erreurs.

## 8. ANALYSE DES RÉSULTATS

### 8.1 Interprétation

Les résultats obtenus montrent que le modèle MLP (Multi-Layer Perceptron) atteint une très bonne performance sur le dataset MNIST, avec une précision supérieure à 97%. Cependant, une analyse plus fine des prédictions révèle que certaines erreurs persistent, notamment entre des chiffres visuellement proches tels que 3 et 5, 4 et 9, 7 et 1. Ces erreurs sont normales car les chiffres manuscrits varient fortement selon les individus, certaines écritures sont mal centrées ou bruitées et le modèle ne capture pas la structure spatiale des pixels. Donc le MLP est performant, mais limité pour l'analyse d'images complexes.

### 8.2 Forces et Limites

**Forces :** modèle simple et rapide à entraîner, bonne performance globale (>97%), facile à implémenter et interpréter, convergence rapide grâce à Adam optimizer.

**Limites :** ne prend pas en compte la structure spatiale de l'image, perte d'information lors du flatten ( $28 \times 28 \rightarrow 784$ ), moins performant que les CNN pour les images, sensible aux écritures ambiguës.

### 8.3 Analyse des erreurs

L'analyse des erreurs via la matrice de confusion montre que les erreurs ne sont pas aléatoires, elles sont concentrées sur des classes proches visuellement et le modèle confond principalement les chiffres ayant des formes similaires.

**Causes principales :** écriture manuscrite irrégulière, bruit dans les images et limitation du MLP à traiter les images comme vecteurs simples.

## 9. AMÉLIORATIONS

- **Dropout (réduction du surapprentissage)** consiste à désactiver aléatoirement certains neurones pendant l'entraînement.
  - Objectif : éviter que le modèle mémorise les données, améliorer la généralisation.
  - Effet attendu : réduction de l'overfitting, meilleure performance sur données test
- **Utilisation des CNN** (Convolutional Neural Networks) sont beaucoup plus adaptés aux images car ils conservent la structure 2D de l'image, détectent les motifs locaux (bords, formes) et sont invariants aux translations.
- **Augmentation des données** consiste à augmenter artificiellement les données (rotation, zoom, translation, bruit)
  - Avantages : améliore la robustesse du modèle et réduit le surapprentissage.
- **Optimisation des hyperparamètres** : les performances du modèle dépendent fortement de learning rate, batch size, nombre d'époques et architecture du réseau
  - Exemple : learning rate ( trop grand → divergence) et ( trop petit → apprentissage lent).
- **Early Stopping** permet d'arrêter l'entraînement lorsque la performance sur validation n'améliore plus.
  - Objectif : éviter le surapprentissage et économiser du temps de calcul.

## **10. CONCLUSION**

Ce projet a permis de concevoir et d'entraîner un modèle de type MLP (Multi-Layer Perceptron) pour la reconnaissance automatique des chiffres manuscrits à partir du dataset MNIST. Les résultats obtenus montrent que le modèle atteint une bonne accuracy (>97%), apprend efficacement les caractéristiques des images et généralise correctement sur les données de test. Cependant, malgré ces performances satisfaisantes, certaines limites importantes ont été identifiées comme perte d'information spatiale lors du flattening, difficulté à distinguer les chiffres visuellement proches et performance inférieure aux modèles convolutifs. Ainsi, ce projet met en évidence que les réseaux MLP sont efficaces pour des problèmes simples, mais deviennent limités pour des tâches de vision par ordinateur. Et pour améliorer les performances, il serait pertinent d'utiliser des architectures plus avancées telles que les CNN (Convolutional Neural Networks) ou les modèles hybrides.

## **11. RÉFÉRENCES**

-  Documentation PyTorch
-  Dataset MNIST
-  Goodfellow et al., Deep Learning

**Lien google colab du projet 4 :**

[https://colab.research.google.com/drive/1jFVz4UOUVEPQEFzz1lqATIZIfwl\\_8CCL?usp=sharing](https://colab.research.google.com/drive/1jFVz4UOUVEPQEFzz1lqATIZIfwl_8CCL?usp=sharing)